

Transport Layer Services

- **Process-to-Process Communication**

- **Addressing: Port Numbers**

- **ICANN Ranges**

Well-known ports

Registered ports

Dynamic ports

- **Encapsulation and Decapsulation**

- **Multiplexing and Demultiplexing**

Transport Layer Services

- Flow Control**

Pushing or Pulling

Flow Control at Transport Layer

Buffers

- Error Control**

Sequence Numbers

Acknowledgment

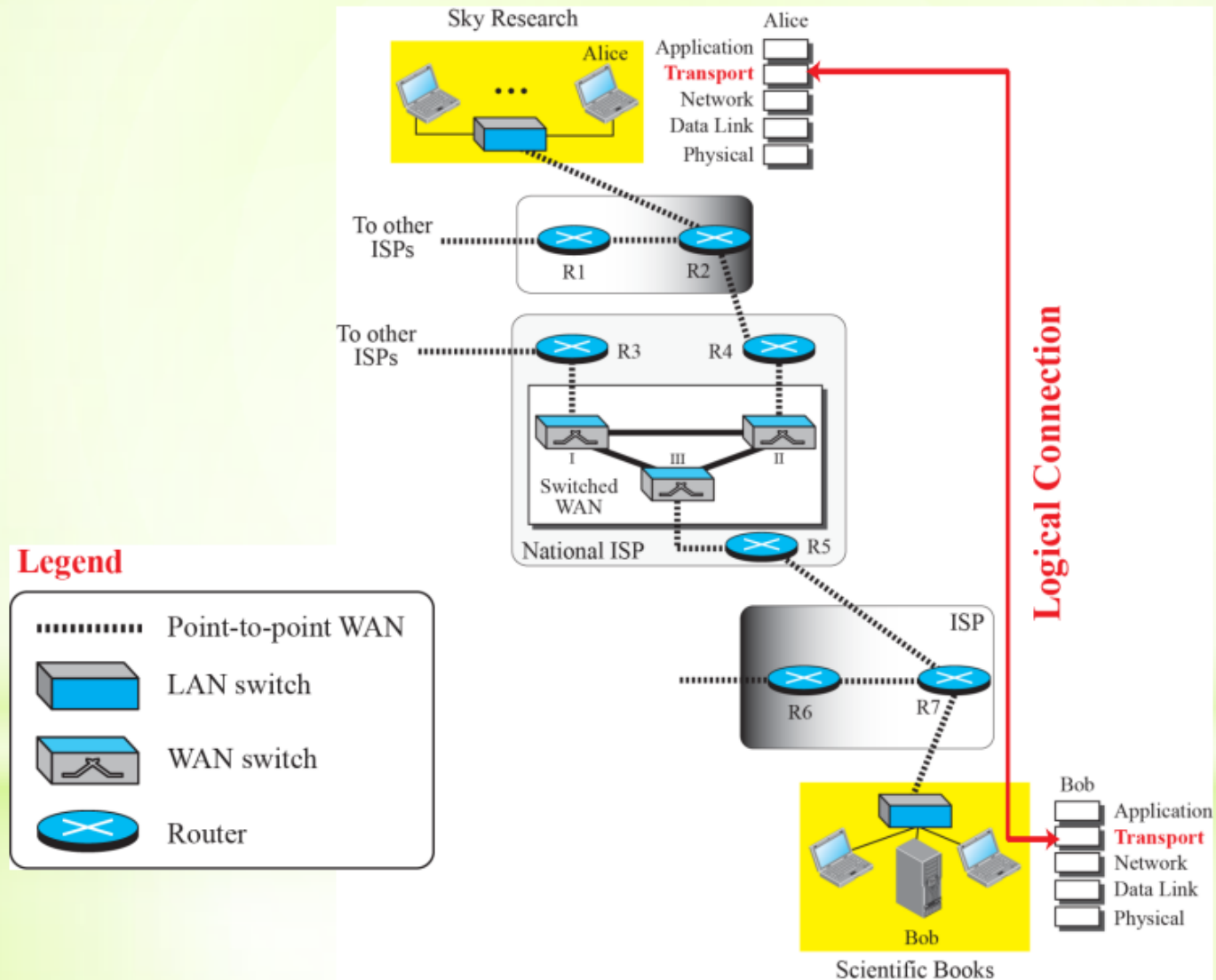
- Combination of Flow and Error Control**

Sliding Window

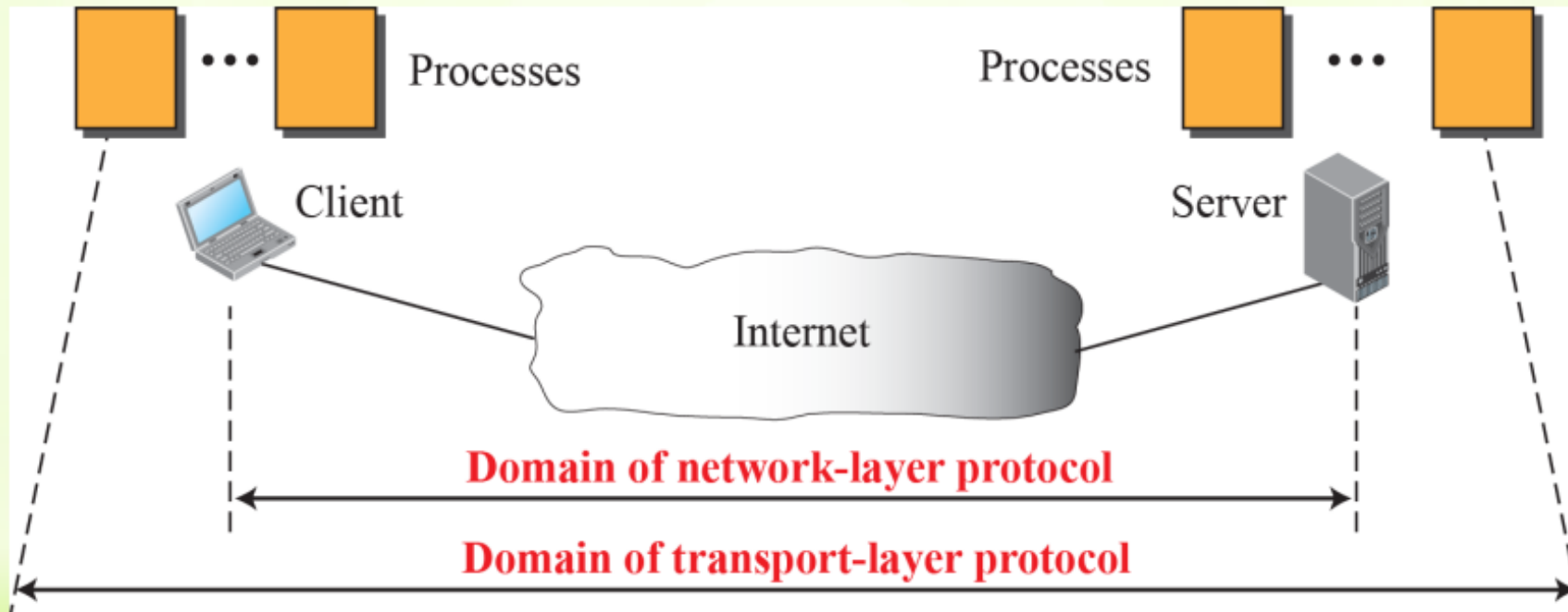
Congestion Control

Connectionless and Connection-Oriented

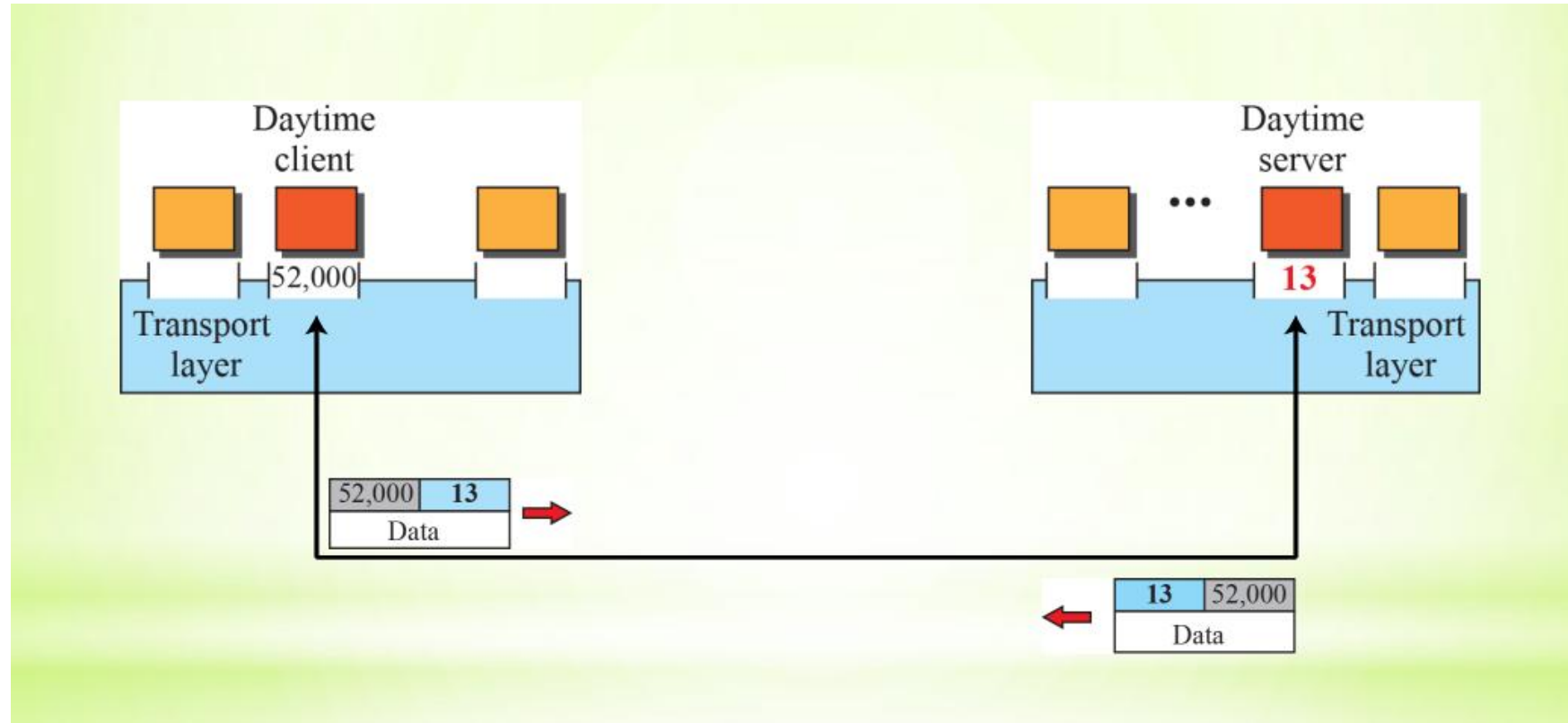
Logical connection at the transport layer



Network layer versus transport layer



Daytime client-server



Step-by-step explanation

•Client side

The client application requests the daytime service.

The transport layer assigns an **ephemeral source port (52,000)** to the client process.

A segment is created with:

Source port: 52,000, **Destination port:** 13, **Data:** request for time

•Server side

The segment arrives at the server.

The transport layer checks the **destination port = 13**.

Since port 13 is bound to the **Daytime server process**, the data is delivered to it.

•Reply from server

The server sends the response (current date and time).

A new segment is created with:

Source port: 13

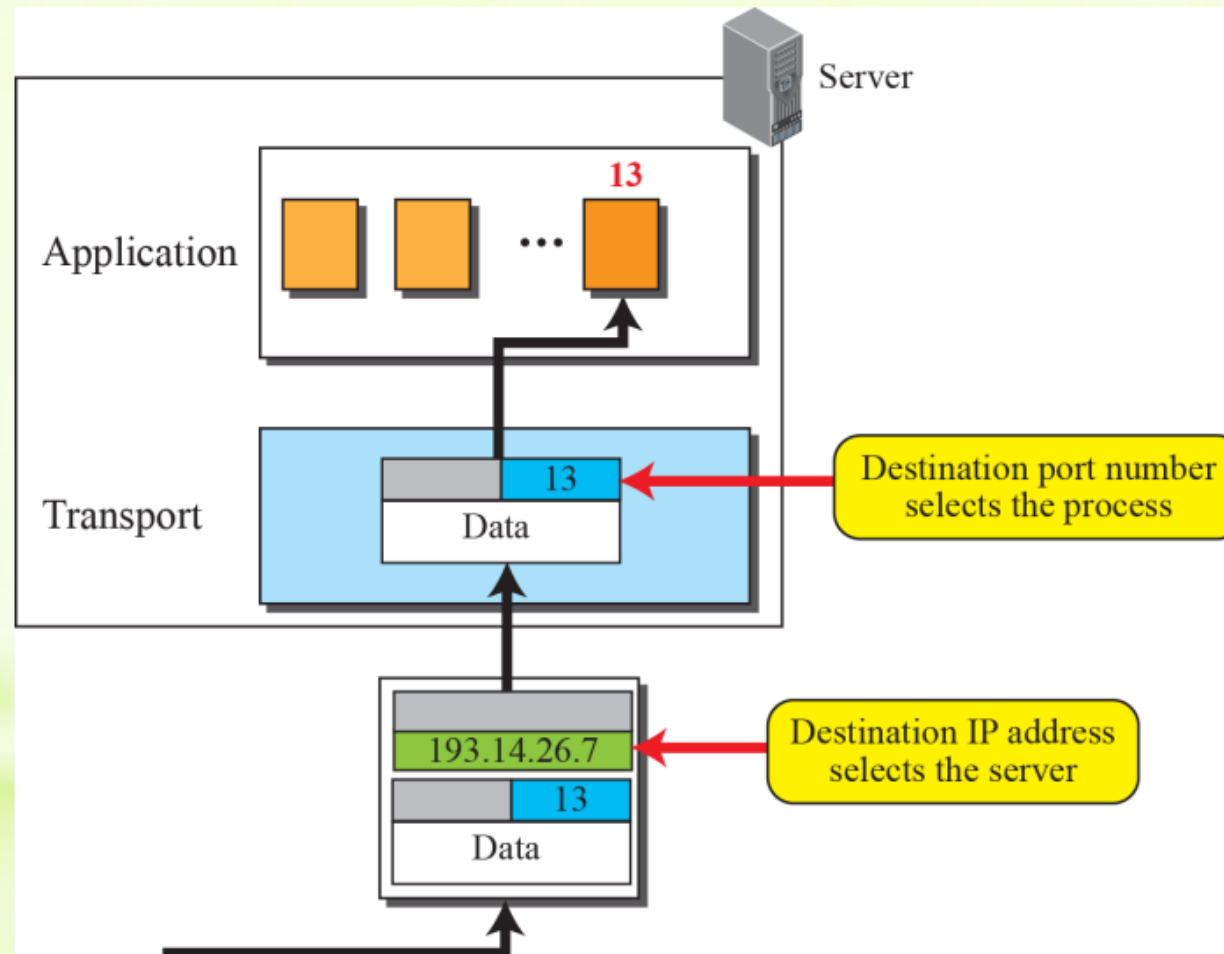
Destination port: 52,000

Data: daytime information

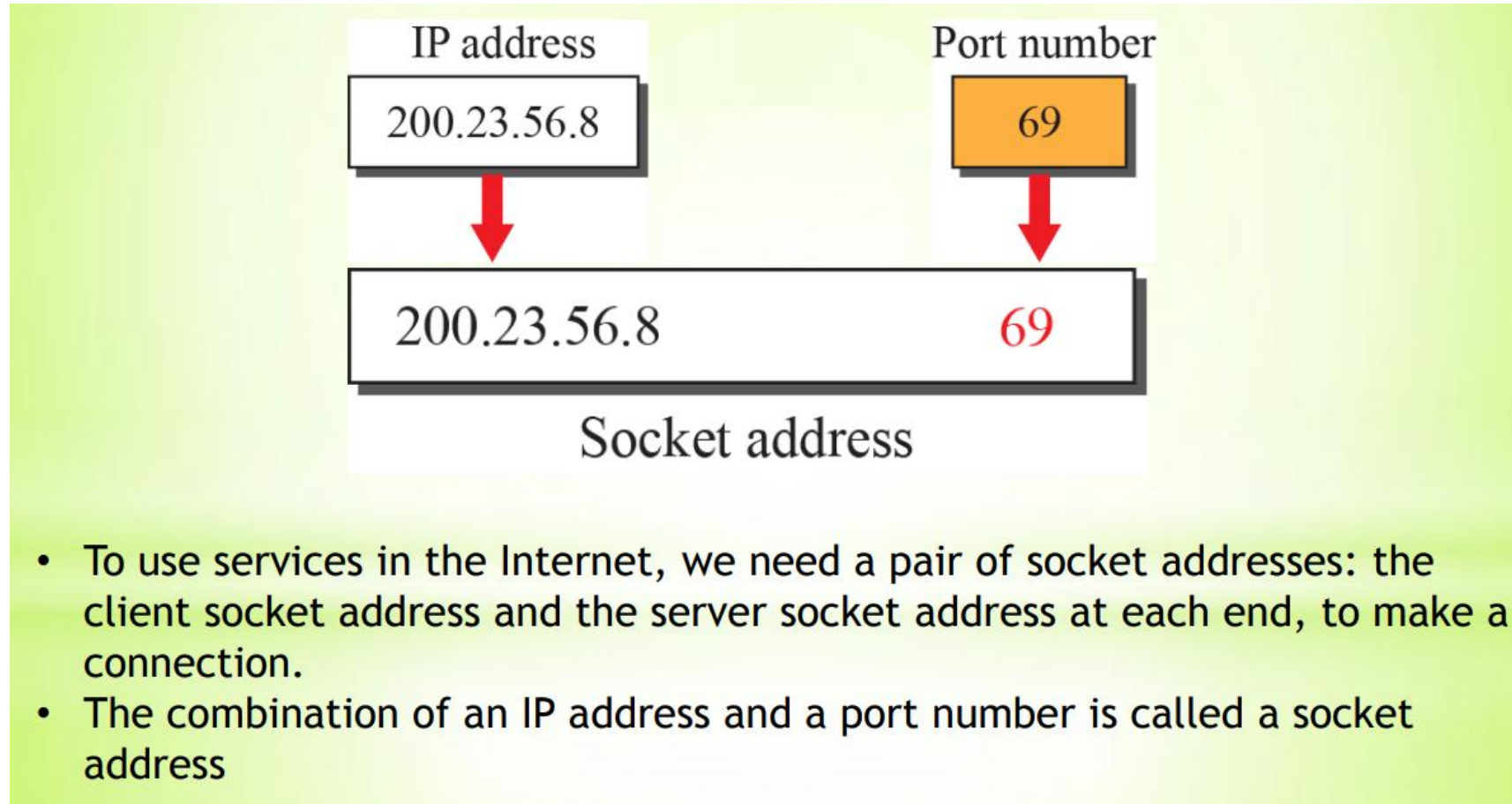
• Client receives response

- The client transport layer uses **port 52,000** to **deliver the data to the correct client process**.

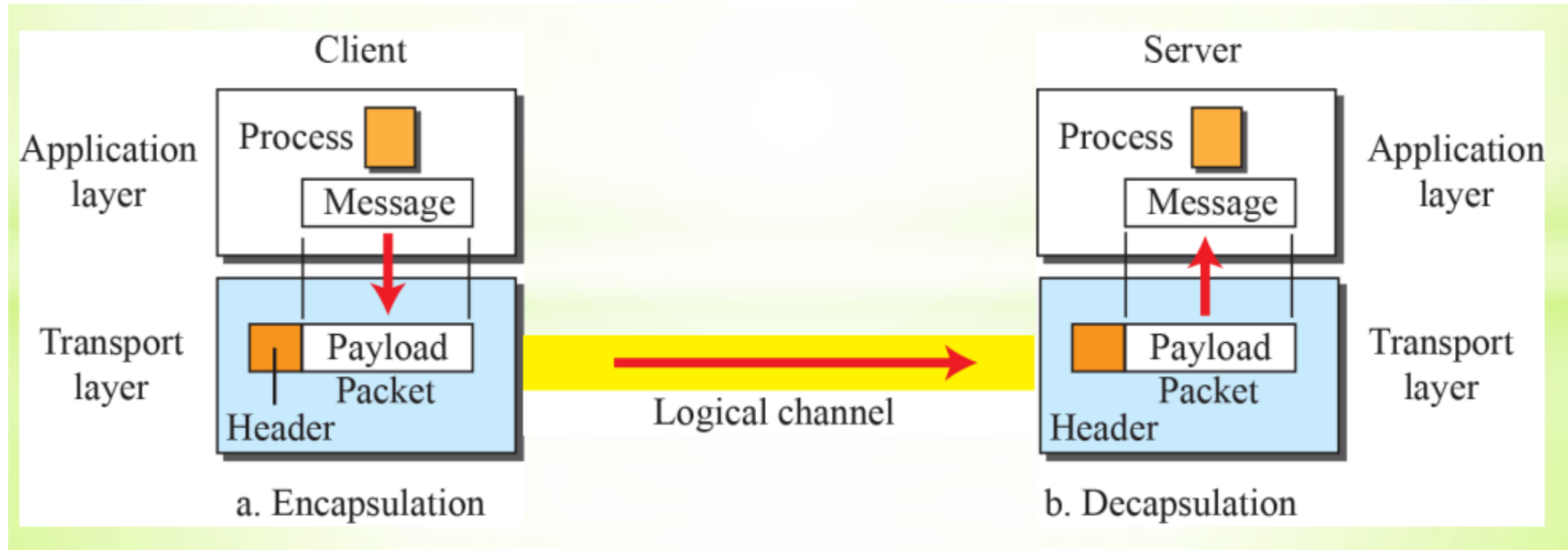
IP addresses versus port numbers



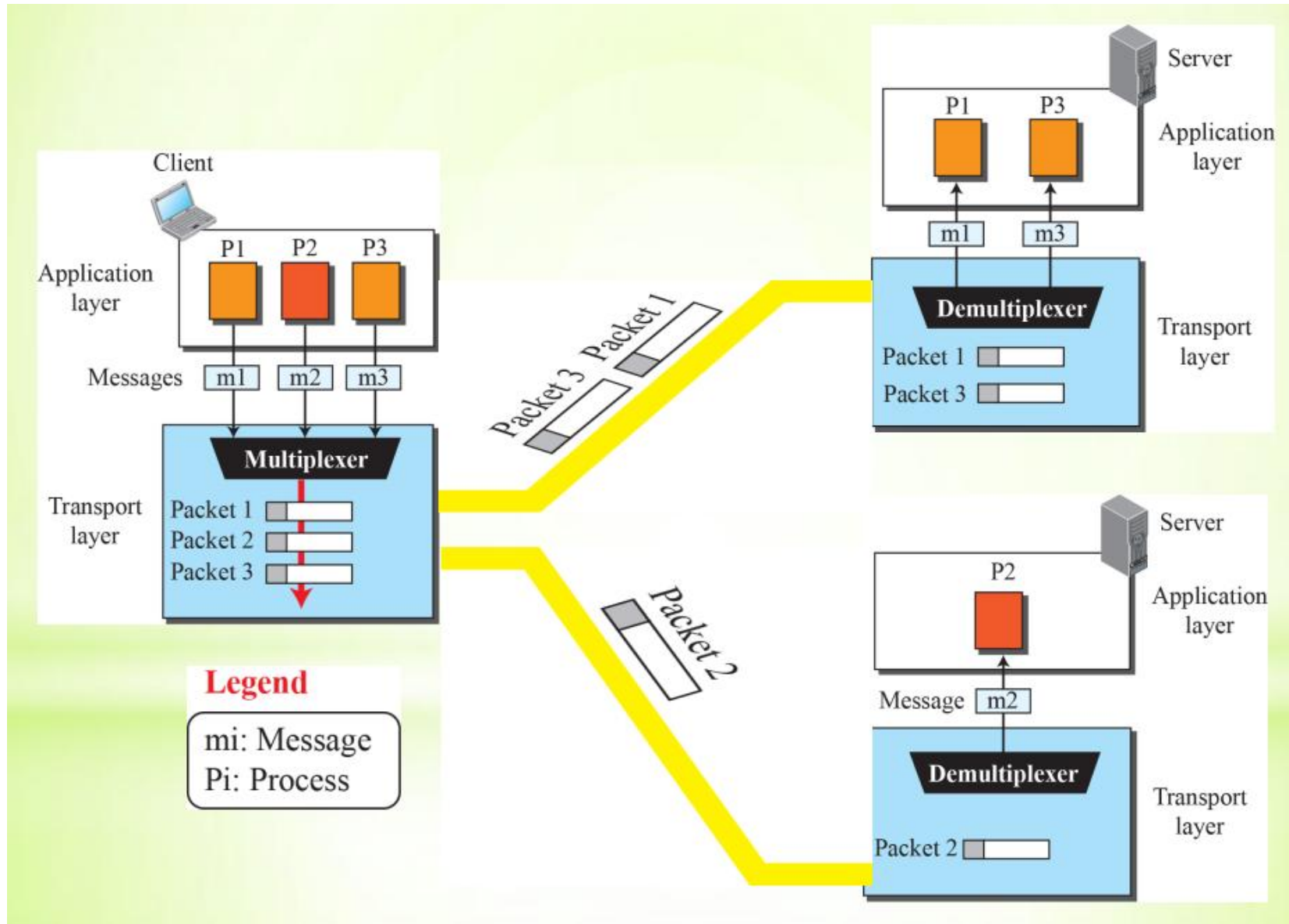
Socket Address



Encapsulation and decapsulation

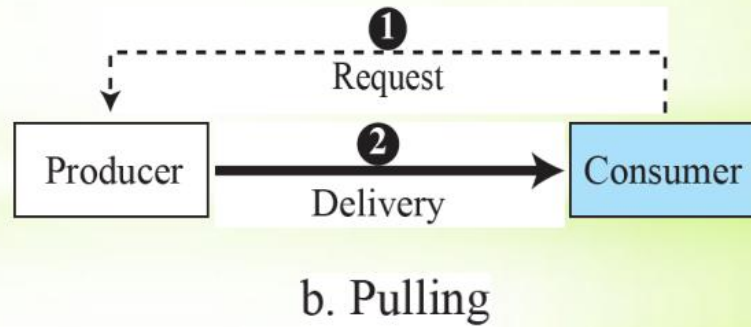
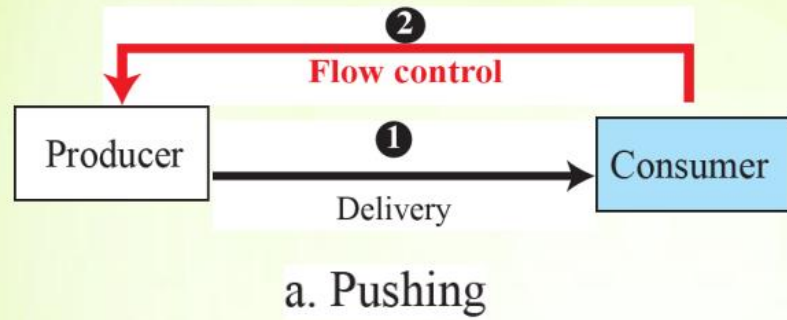


Multiplexing and demultiplexing



Flow Control

Pushing or pulling



Flow Control

a) Pushing

Producer controls the data flow

How it works

The **producer sends (pushes) data** to the consumer whenever it is ready.

The **consumer uses flow control feedback** (e.g., acknowledgments, window size, “slow down” signals) to regulate the sender.

Data delivery happens **first**

Control information flows **backward** from consumer to producer

Example

TCP: Sender pushes data, receiver controls the rate using the window size.

Streaming where the sender continuously sends data.

Efficient for continuous data streams

Flow Control

b) Pulling

Consumer controls the data flow

How it works

1. The **consumer requests data** when it is ready.
2. The **producer delivers data only after the request.**

Control comes before delivery

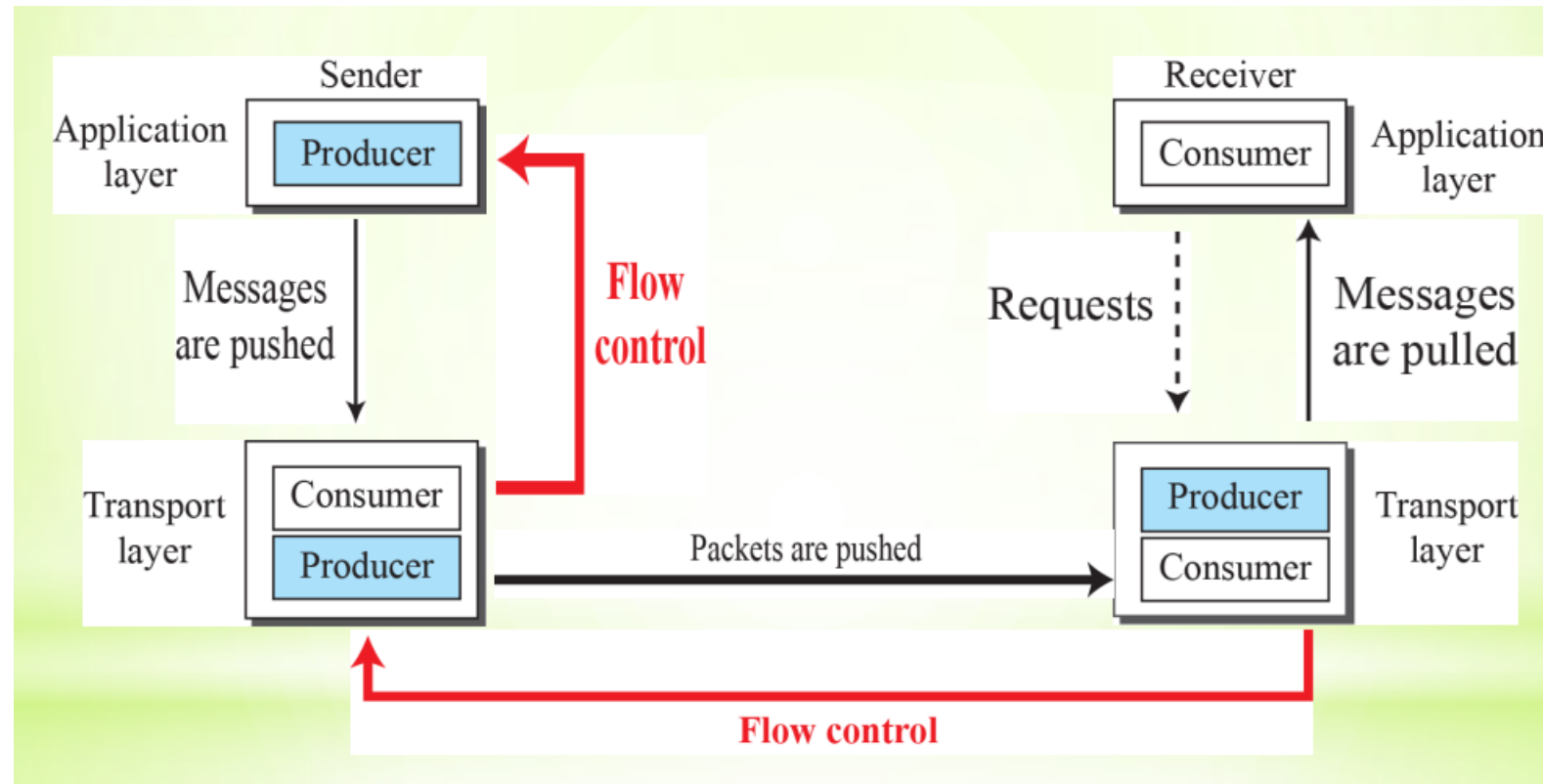
The receiver explicitly decides *when* data is sent

Example

- HTTP request–response model
- Database queries

The receiver never gets overloaded

Flow Control



SEQUENCE & ACKNOWLEDGMENT NUMBERS

- To perform error control, the packets are numbered. We can add a field to the transport-layer packet to store the packet's sequence number. Packets are numbered sequentially.

If the packet header allows m bits for the sequence number, the sequence numbers range from 0 to $2^m - 1$.

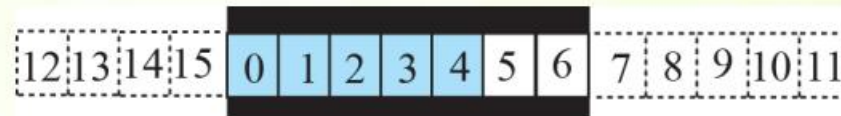
- We can use both positive and negative signals as error control.
- The receiver side can send an acknowledgement (ACK) for each of a collection of packets that have arrived safe and sound. The receiver can simply discard the corrupted packets.

SLIDING WINDOW

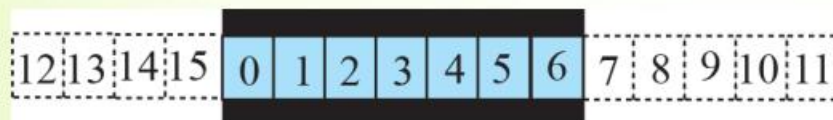
Sliding window in linear format



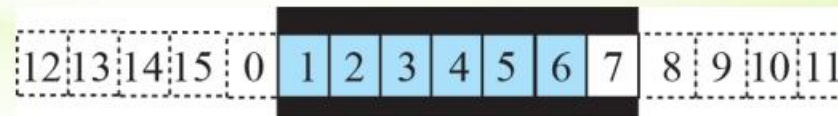
a. Four packets have been sent.



b. Five packets have been sent.

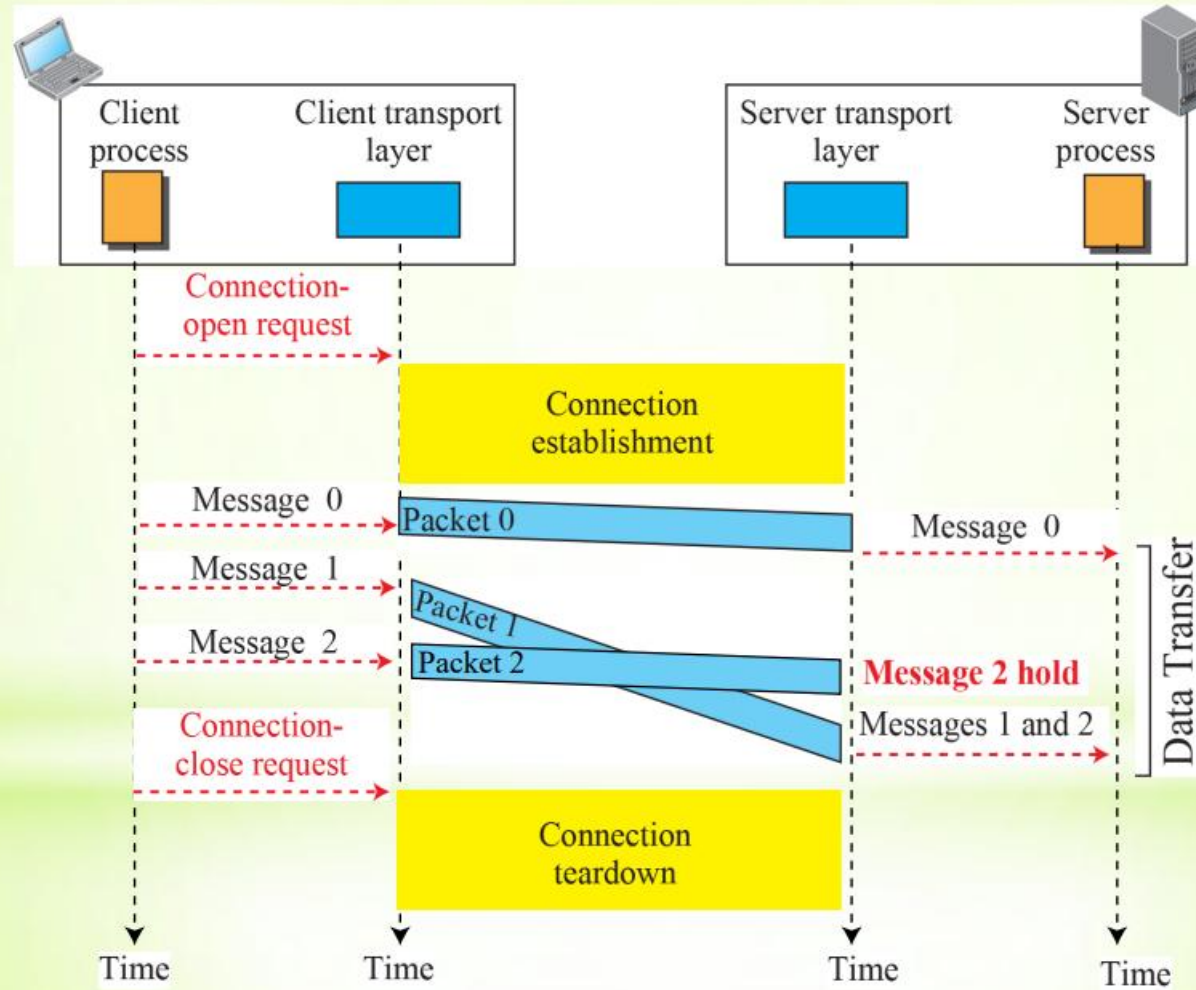


c. Seven packets have been sent;
window is full.



d. Packet 0 has been acknowledged;
window slides.

Connection-oriented service



Transmission Control Protocol (TCP)

TCP is a connection-oriented protocol.

It establishes a virtual connection between two TCP endpoints to send data.

TCP uses flow- and error-control mechanisms at the transport level.

The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with a randomly generated number.

Transmission Control Protocol (TCP)

Example : Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?

Segment 1	➡	Sequence Number: 10,001 (range: 10,001 to 11,000)
Segment 2	➡	Sequence Number: 11,001 (range: 11,001 to 12,000)
Segment 3	➡	Sequence Number: 12,001 (range: 12,001 to 13,000)
Segment 4	➡	Sequence Number: 13,001 (range: 13,001 to 14,000)
Segment 5	➡	Sequence Number: 14,001 (range: 14,001 to 15,000)

Transmission Control Protocol (TCP)

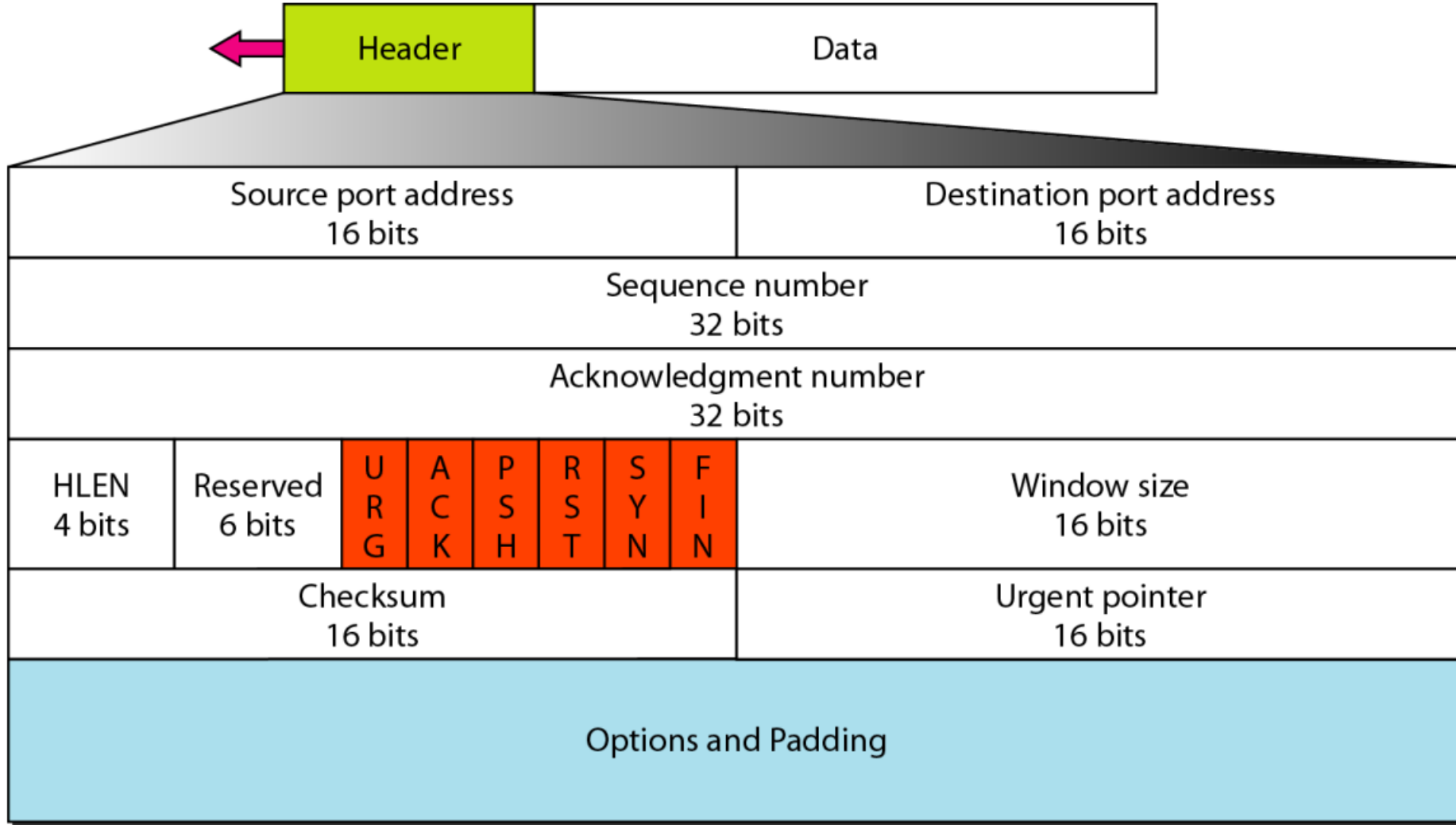
The value in the sequence number field of a segment defines the number of the first data byte contained in that segment.

Transmission Control Protocol (TCP)

**The value of the acknowledgment field
in a segment defines
the number of the next byte a party
expects to receive.**

**The acknowledgment number is
cumulative.**

TCP Header



TCP Header

TCP Header Format (Minimum 20 Bytes)

The TCP header consists of the following fields, ensuring ordered and error-free data delivery:

- **Source Port (16 bits):** Identifies the sending application.
- **Destination Port (16 bits):** Identifies the receiving application.
- **Sequence Number (32 bits):** Tracks the byte stream position.
- **Acknowledgement Number (32 bits):** Confirms receipt of bytes.
- **Data Offset (4 bits):** Indicates the header size (start of data).
- **Reserved (3 bits):** Set to zero.
- **Flags (9 bits):** Control bits (URG, ACK, PSH, RST, SYN, FIN).
- **Window Size (16 bits):** Used for flow control.
- **Checksum (16 bits):** Error checking for header and data.
- **Urgent Pointer (16 bits):** Indicates urgent data if the URG flag is set.
- **Options (0–40 bytes):** Optional parameters, padding ensures a 32-bit boundary.

TCP Header Control Feilds

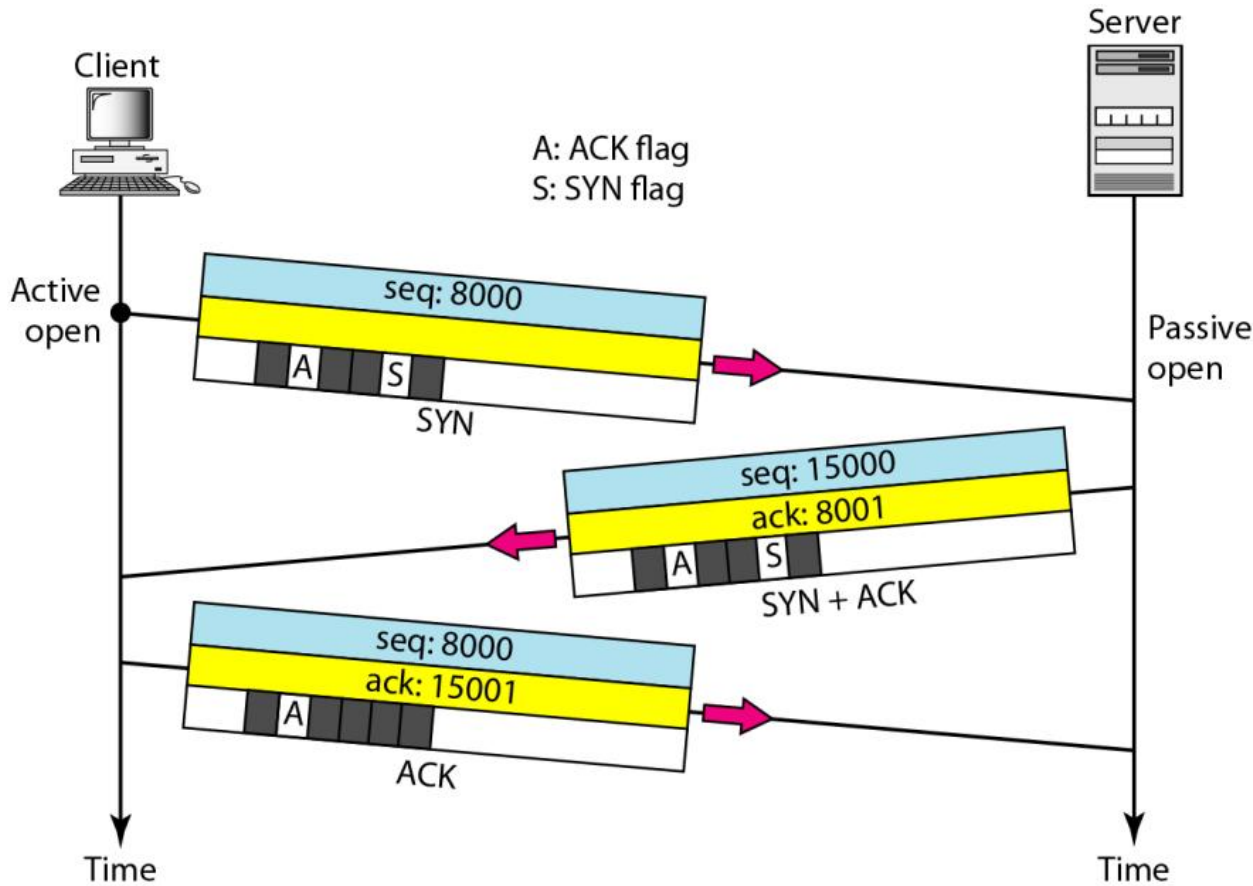
URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



<i>Flag</i>	<i>Description</i>
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

TCP Connection establishment using three-way handshaking



Step 1 – SYN (Active Open):

The client initiates the connection by sending a segment with the **SYN flag set** and a sequence number (seq = 8000). This indicates a request to establish a connection.

Step 2 – SYN + ACK (Passive Open):

The server responds with a segment that has both the **SYN and ACK flags set**, its own sequence number (seq = 15000), and an acknowledgement number (ack = 8001).

This acknowledges the client's SYN (8000 + 1).

Step 3 – ACK:

The client sends a final segment with the **ACK flag set**, acknowledging the server's sequence number (ack = 15001).

This confirms receipt of the server's SYN (15000 + 1).

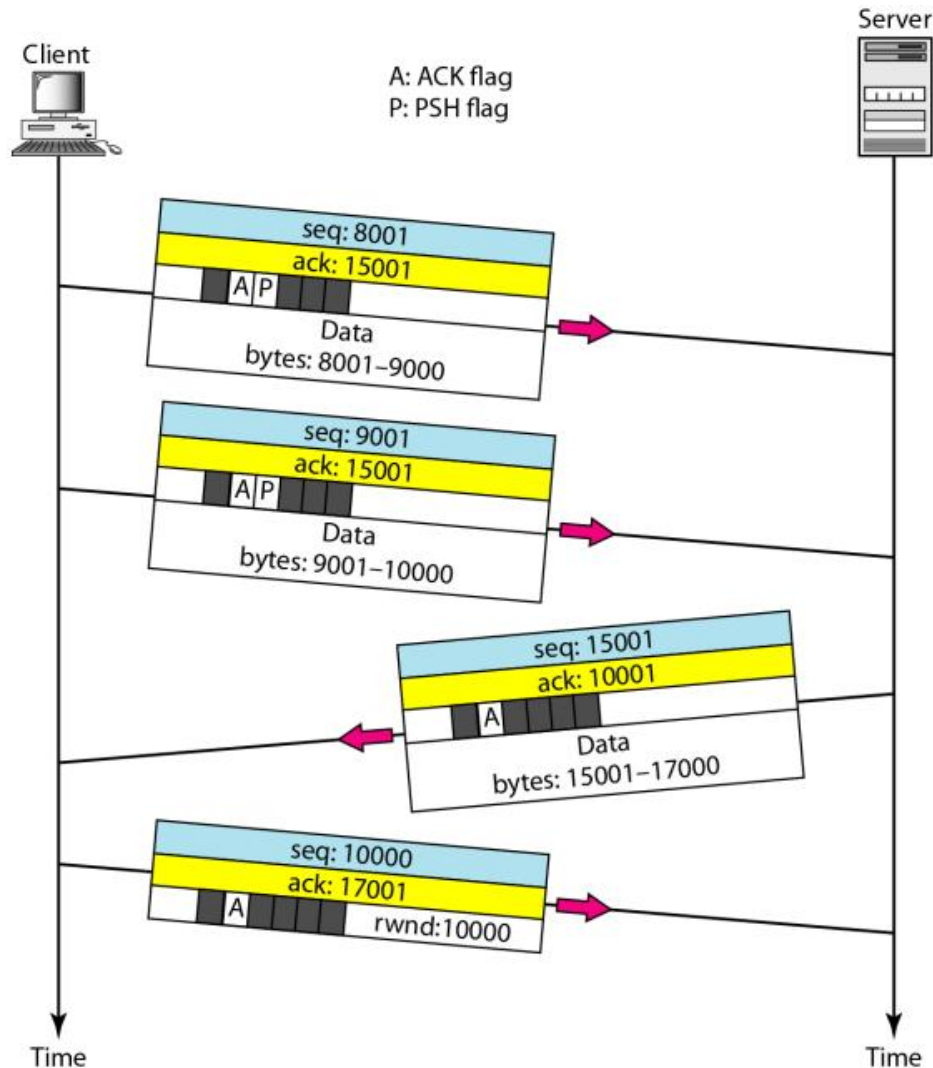
A SYN segment cannot carry data, but it consumes one sequence number.

A SYN + ACK segment cannot carry data, but does consume one sequence number.

An ACK segment, if carrying no data, consumes no sequence number.

TCP Data transfer in three-way handshaking

Fig. demonstrating **bidirectional communication, acknowledgement mechanism, and flow control**.



1 Client → Server Data Transmission

- The client sends data in two segments:
 - Segment 1:** seq = 8001, ack = 15001, data bytes 8001-9000
 - Segment 2:** seq = 9001, ack = 15001, data bytes 9001-10000
- The ACK value (15001) indicates the client expects byte 15001 from the server.
- The **PSH flag** indicates data should be pushed to the application immediately.

2 Server → Client Data & Acknowledgment

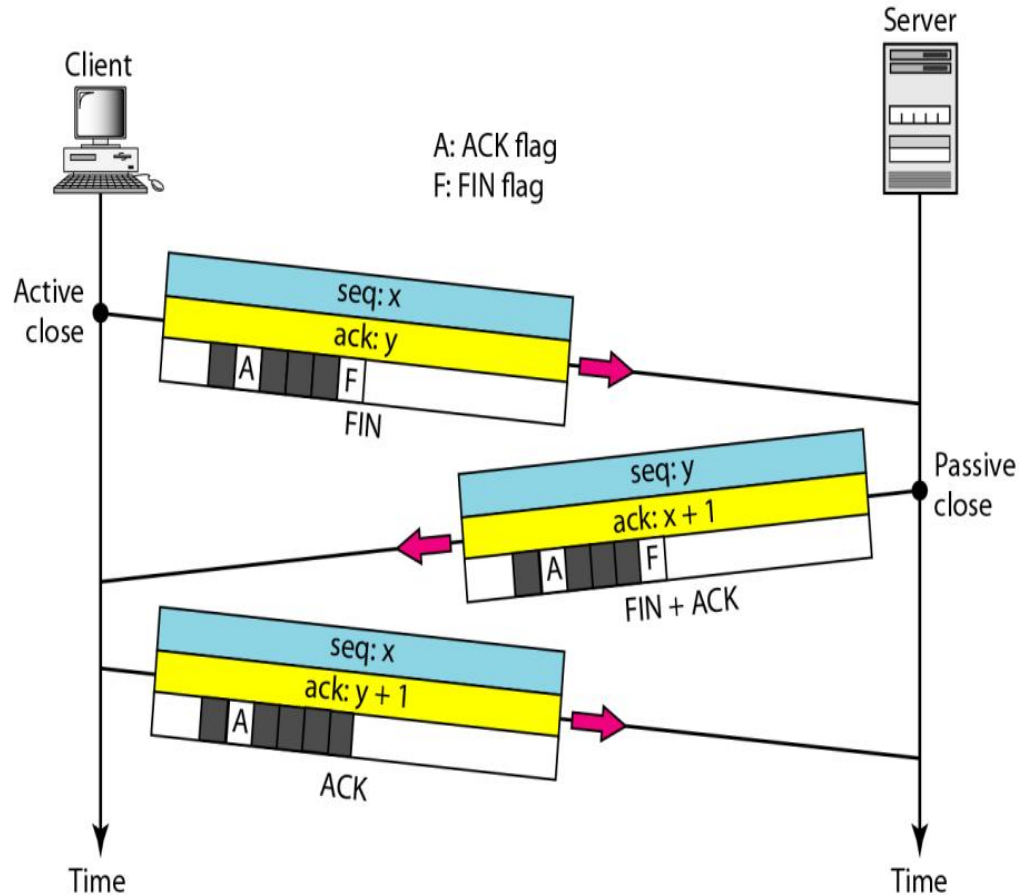
- The server sends:
 - seq = 15001, ack = 10001
 - Data bytes 15001-17000
- ack = 10001 confirms receipt of client bytes up to 10000 (next expected byte is 10001).

3 Final Acknowledgment & Flow Control

- The client sends:
 - ack = 17001
 - rwnd = 10000
- This acknowledges receipt of server bytes up to 17000.
- The **receiver window (rwnd)** indicates available buffer space for flow control.

TCP Connection termination using three-way handshaking

The figure illustrates the **TCP connection termination process**, showing how a connection is closed via a **FIN-ACK exchange** between the client and the server.



1 Active Close (Client)

- The client initiates termination by sending a segment with the **FIN flag set**.
- Segment details: `seq = x`, `ack = y`
- This indicates that the client has finished sending data.

2 Passive Close (Server)

- The server acknowledges the client's FIN by sending:
 - `seq = y`
 - `ack = x + 1`
 - Flags: **FIN + ACK**
- `ack = x + 1` confirms receipt of the client's FIN (FIN consumes one sequence number).
- The server also sends its own FIN to indicate it has finished sending data.

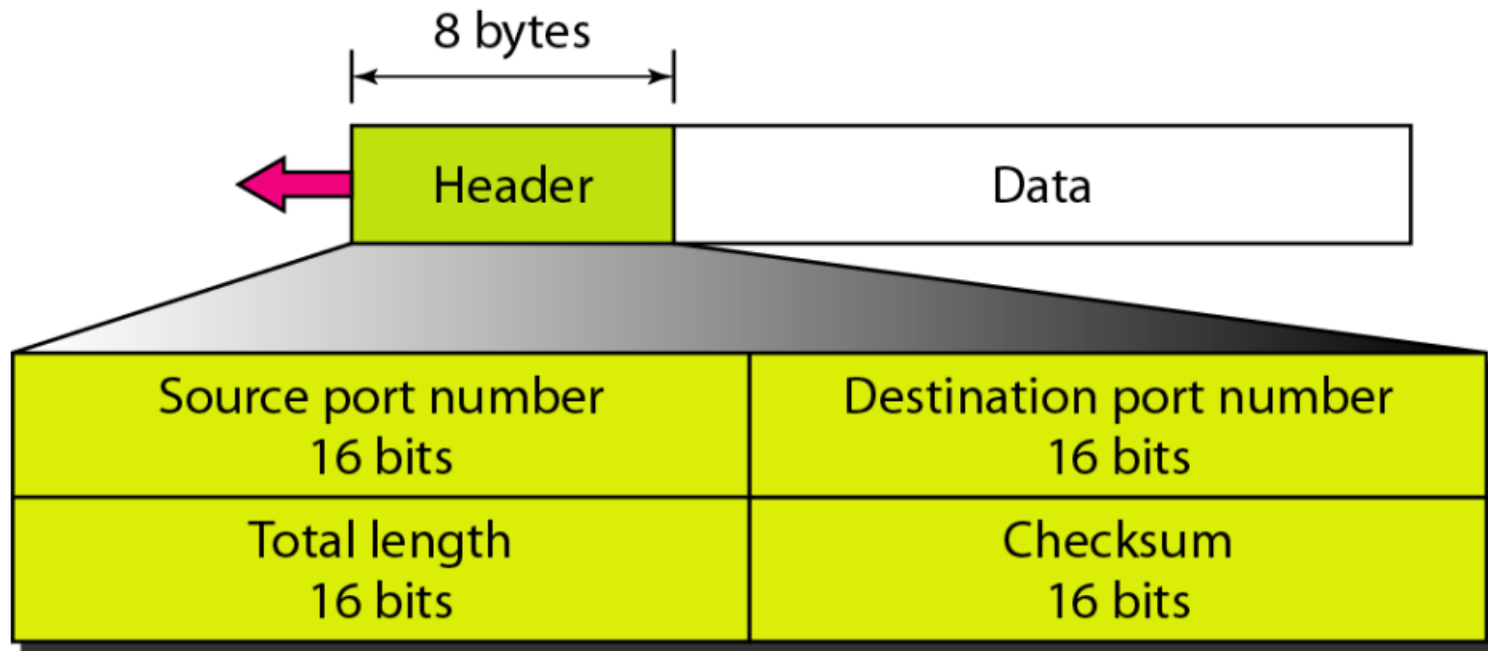
3 Final Acknowledgment (Client)

- The client sends the final **ACK**:
 - `seq = x`
 - `ack = y + 1`
- This acknowledges the server's FIN (again, FIN consumes one sequence number).

The FIN segment consumes one sequence number if it does not carry data.

The FIN + ACK segment consumes one sequence number if it does not carry data.

USER DATAGRAM PROTOCOL (UDP)Header



USER DATAGRAM PROTOCOL (UDP)Header

UDP Header Format (Fixed 8 Bytes)

The UDP header is designed for minimal overhead, containing only four fields:

- **Source Port (16 bits):** Sender's port number.
- **Destination Port (16 bits):** Receiver's port number.
- **Length (16 bits):** Total length of the UDP header and data.
- **Checksum (16 bits):** Error checking for header and payload (optional in IPv4, mandatory in IPv6).

Key Differences

- **Size:** TCP is 20–60 bytes, whereas UDP is always 8 bytes.
- **Reliability:** TCP includes Sequence/ACK numbers; UDP does not.
- **Complexity:** TCP handles flow/congestion control; UDP is connectionless.

USER DATAGRAM PROTOCOL (UDP)Header

Example 1 :The following is a dump of a UDP header in hexadecimal format.

CB84000D001C001C

- a. What is the source port number?
- b. What is the destination port number?
- c. What is the total length of the user datagram?
- d. What is the length of the data?